Corte #3 de Proyecto - Medinet

Ingeniería de Software 1

Lynette García

Licenciatura en Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Guatemala, febrero del 2026

Luis Pedro Figueroa #24087

Luis Pedro Hernández #24337

Diego Gudiel #24451

Adair Velásquez #24596

Santiago Cordero #24472

Tabla de Contenidos

Tabla de Contenidos 2
Resumen 5
Introducción 6
Design Thinking 7
Prototipos 7
Versión 1 7
Evidencias de muestra a usuarios 8
Lista de cambios sugeridos 10
Versión 2 11
Lista de cambios sugeridos 14
Cambios a lista de requisitos funcionales 14
Análisis 15
Lista final de requisitos funcionales 15
Clases preliminares 17
Diagrama de clases 17
Descripción de las clases 18
Model: 18
Repositories: 19
Services: 19
Controllers: 20
Routes 20
Diagrama de paquetes 22
Descripción de los paquetes 22
Auth: 22
Users: 22
Clinics: 23
Schedules: 23
Appointment-requests: 23
Waitlists: 23

Notifications:.....23
Persistencia de datos.....23
Selección de tipo de base de datos.....24
Base de datos relacional (SQL): 24
Base de datos documental (NoSQL):.....24
Base de datos en grafo 24
Selección para MediNet:.....25
Diagrama de clases persistentes 26
Diagrama entidad-relación 27
Diseño 28
Selección de tecnología.....28
Estimaciones de dimensiones del sistema 28
Cantidad de usuarios totales: 28
Tiempo promedio de respuesta: 28
Cantidad de usuarios simultáneos:.....29
Tamaño en disco: 29
Disponibilidad de la información: 30
Confiabilidad o tolerancia a fallos: 30
Tecnología por utilizar.....31
Framework y Lenguaje de programación: 31
Frontend:.....33
Arquitectura: 35
Control de versiones: 37
Selección final:.....38
Gestor de base de datos 40
Selección final:.....41
Planificación e informe de gestión.....42
Informe de gestión.....42
Conclusión de desempeño.....44
Formularios LOGT.....44
Bibliografía 48

Resumen

El presente trabajo describe el desarrollo del sistema MediNet, una plataforma digital orientada a centralizar y gestionar la programación de citas médicas mediante una agenda única en línea. El proyecto surge ante la necesidad actual de reducir errores, conflictos de horario y desorganización causados por el uso de múltiples herramientas de agendación, así como mejorar la comunicación entre médicos, personal administrativo y pacientes. Esta problemática es relevante porque afecta directamente la eficiencia operativa de las clínicas y la experiencia del paciente, generando pérdidas de tiempo, duplicidad de citas y falta de información actualizada.

El objetivo principal del trabajo es diseñar e implementar un sistema centralizado que permita visualizar y gestionar citas en tiempo real, evitando traslapes y facilitando la coordinación entre múltiples clínicas. Además, se busca simplificar el proceso de registro y agendamiento para los pacientes, automatizar notificaciones sobre cancelaciones o reprogramaciones, y ofrecer disponibilidad de horarios actualizada. Con ello, el sistema pretende optimizar la organización médica, mejorar la comunicación con los usuarios y garantizar un proceso de agendación más rápido, confiable y eficiente para todos los involucrados.

Introducción

El proyecto por realizar está dirigido al entorno de atención médica donde participan médicos, personal administrativo (secretarias) y pacientes, quienes interactúan constantemente en el proceso de programación y gestión de citas. La entidad descrita se centra en la prestación de servicios médicos mediante consultas programadas, control de horarios clínicos y administración de información de pacientes. Dentro de su estructura, el área principal involucrada en el proyecto es la gestión de citas médicas, la cual coordina la disponibilidad del médico, la comunicación con pacientes y la actualización continua de horarios, siendo un proceso crítico para el funcionamiento eficiente de la clínica.

La idea del proyecto consiste en el desarrollo de MediNet, una plataforma digital centralizada que permita administrar citas médicas en una única agenda sincronizada. La problemática surge debido al uso de múltiples herramientas de agendación, lo que genera conflictos de horario, duplicidad de citas, falta de comunicación entre clínicas y procesos manuales ineficientes. Esto provoca desorganización, pérdida de tiempo y errores que afectan tanto al personal médico como a los pacientes. La solución propuesta busca integrar todos los métodos de agendación en un solo sistema, permitiendo actualizaciones en tiempo real, notificaciones automáticas y visualización clara de disponibilidad de horarios.

El objetivo general de este informe es describir el análisis, diseño y planteamiento del sistema MediNet como solución tecnológica para la gestión eficiente de citas médicas. Como objetivos específicos, se pretende: analizar la problemática del proceso actual de agendación, definir los requerimientos y funcionalidades del sistema, establecer la modelación del sistema, y presentar los elementos clave que permitan construir una solución que optimice la organización de citas, mejore la comunicación entre actores y reduzca errores operativos.

Design Thinking

Prototipos

Versión 1

Figura 1. Prototipo vista administrativa v1.

Figura 2. Prototipo vista de paciente v1.

Evidencias de muestra a usuarios

Figura 3. Doctor. Figueroa interactuando con la vista administrativa del prototipo v1.

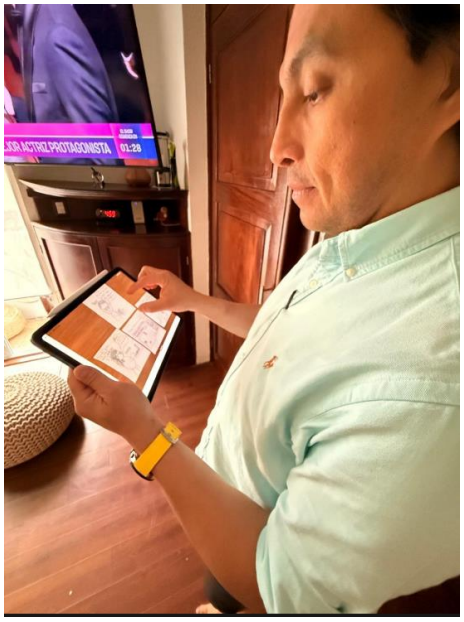


Figura 4. Doctor Figueroa interactuando con la vista de paciente de prototipo v1.

Figura 5. Doctora Monroy interactuando con la vista administrativa paciente v1.

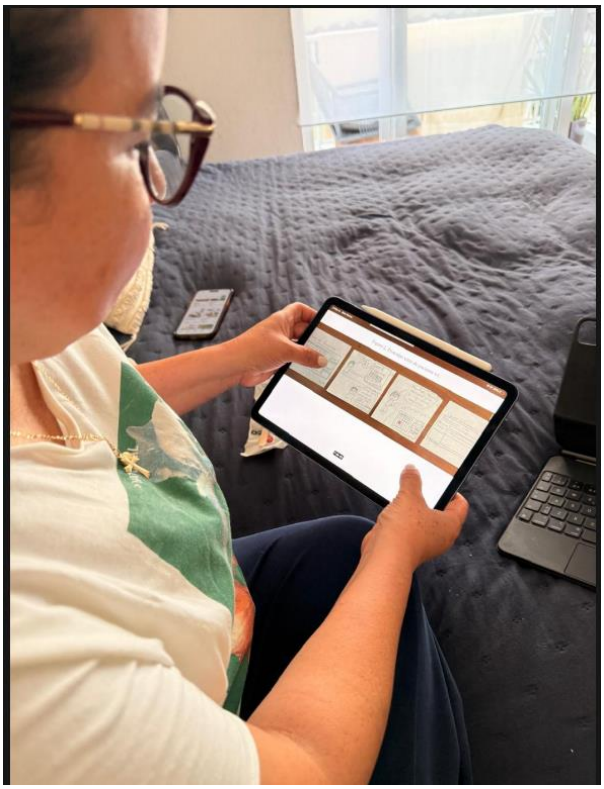
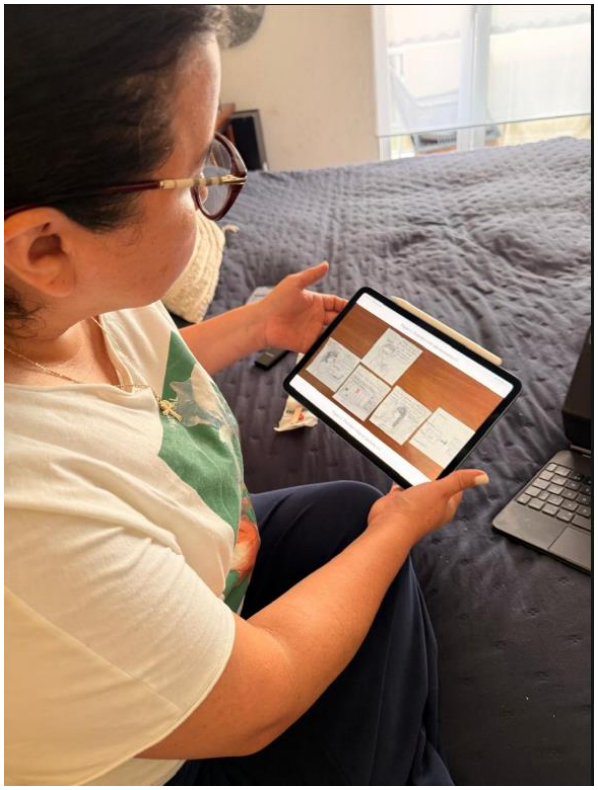


Figura 6. Doctora Monroy interactuando con la vista paciente v1.



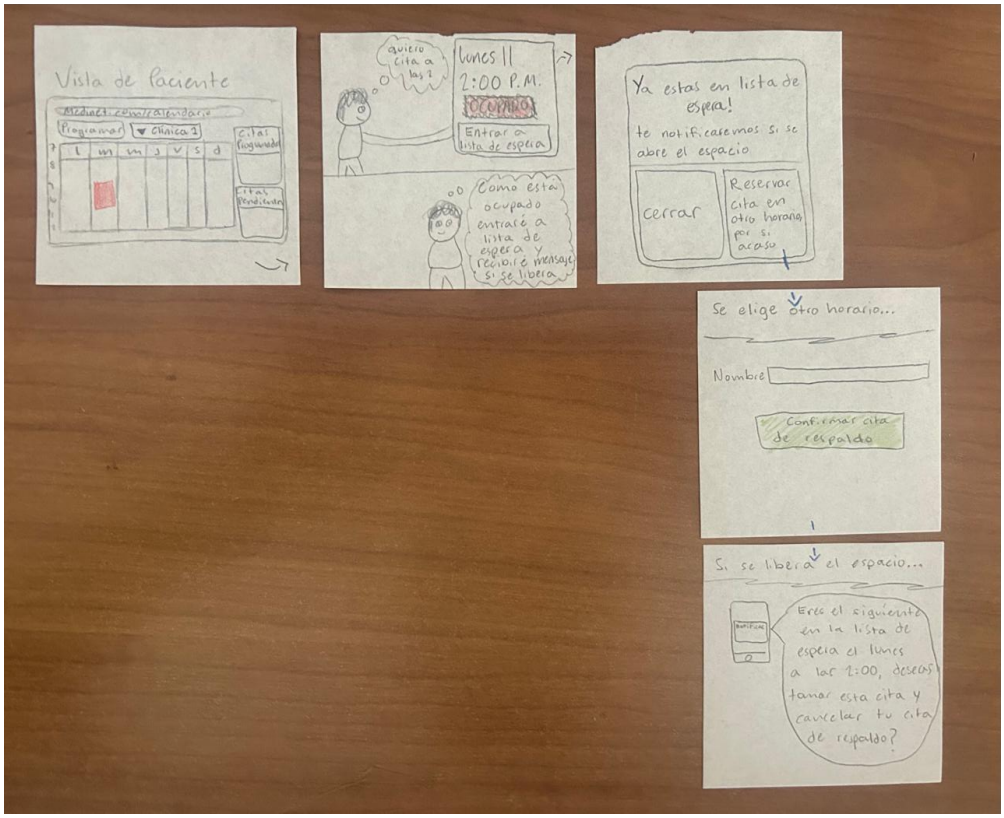
Lista de cambios sugeridos

- El doctor Giovanni Figueroa mencionó que a él no le sirve tener información extensa del paciente para poder realizar una cita. El doctor indicó que lo que más le importa es que haya sincronización de todas las citas entre secretarias y clínicas.
- La secretaria Irene Ortiz mencionó que le encantaría que, al entrar a lista de espera para una cita en un horario, el usuario pueda reservar una cita en otro momento, como fallback, que esté linkeada a la lista de espera. De esta manera, si se liberará el horario deseado y se reservara la cita para la cual estaba esperando, la otra cita se cancelara automáticamente y así se evitarían citas fantasma que hacen parecer que la clínica no tiene espacio cuando en realidad si lo tiene.

Versión 2

Figura 7. Prototipo vista administrativa v2.

Figura 8. Prototipo vista de paciente v2.



Evidencia de muestra a usuarios

Figura 9. Doctor Figueroa interacuando con la vista administrativa prototipo v2

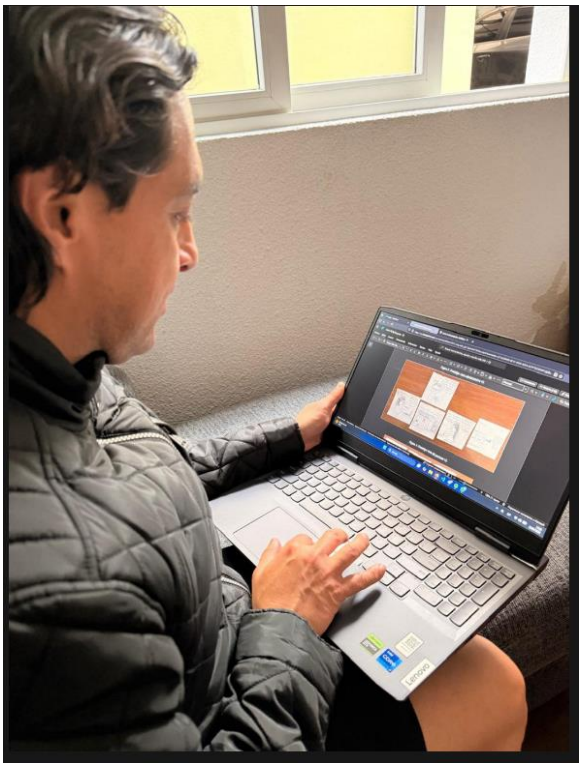


Figura 10. Doctor Figueroa interactuando con la vista de paciente prototipo v2

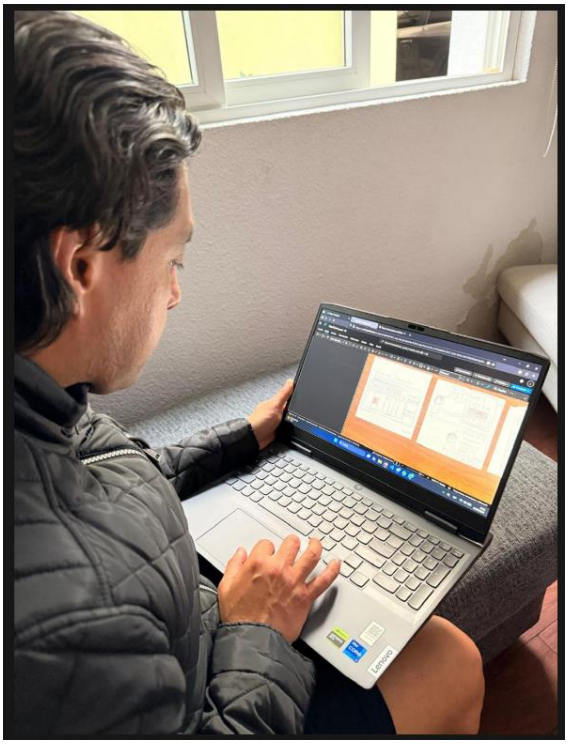


Figura 11. Doctora Monroy interactuando con la vista administrativa prototipo v2

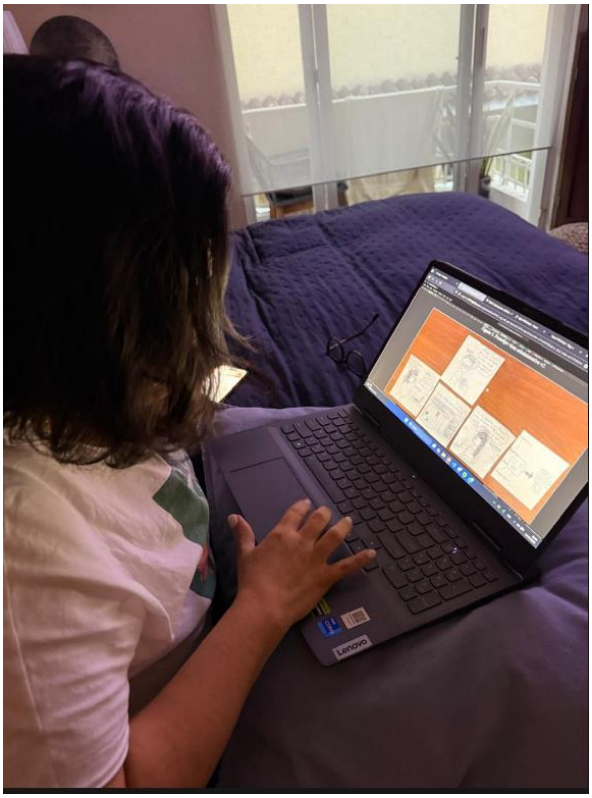
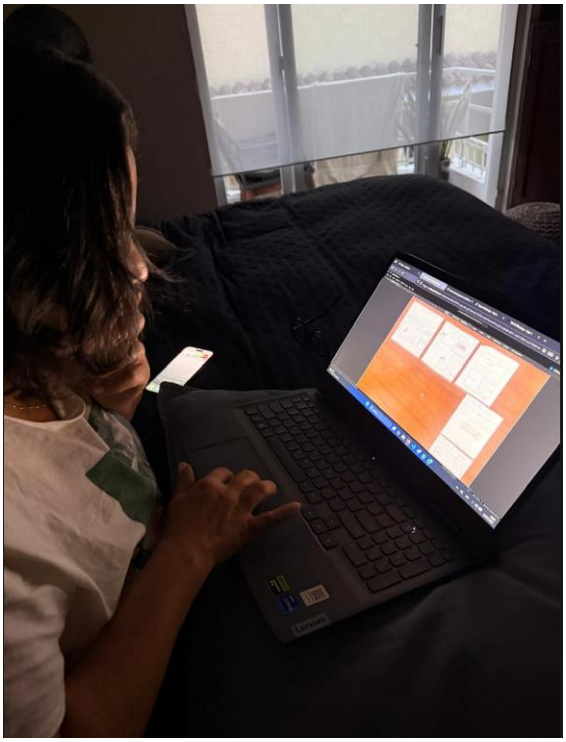


Figura 12. Doctora Monroy interactuando con la vista paciente prototipo v2



Lista de cambios sugeridos

- Al haber hecho el segundo prototipo con apoyo muy cercano de dos doctores y una secretaria, ya no se sugirieron más cambios al prototipo, de hecho, los usuarios mencionaron que la solución les parecía muy buena.

Cambios a lista de requisitos funcionales

1. Se elimina la funcionalidad de guardado de expedientes médicos, por recomendación del doctor. Con esto también se elimina la posibilidad de implementar un autocompletado de información personal del paciente que facilitaría el llenado de estos expedientes.
2. Se agrega la funcionalidad de cita de respaldo, en donde una cita está directamente relacionada con otra y puede sufrir cambios automáticos dependiendo de cambios a la otra cita mencionada.

Análisis

Lista final de requisitos funcionales

1. **Gestión centralizada de citas**
    - **Descripción**: El sistema debe permitir crear, modificar y cancelar citas médicas en una agenda única sincronizada para evitar la duplicidad y desorganización.
    - **Historia de Usuario (US1 y US2)**: Como secretaria/doctor, quiero manejar las citas médicas para organizar el horario de manera eficiente y no depender de múltiples herramientas.
2. **Creación de clínicas y sincronización multi-clínica**
    - **Descripción**: El sistema debe permitir crear y centralizar las agendas de las distintas clínicas donde trabaje un médico, mostrando un calendario unificado (aunque distinguiendo las citas de cada clínica) que prevenga traslapes de horario.
    - **Historia de Usuario (US4 y US12)**: Como doctor, quiero ver mis horarios de todas las clínicas en un solo lugar para dejar de preocuparme por conflictos de horario entre ellas.
3. **Gestión de cuentas y roles**
    - **Descripción**: El sistema debe permitir la creación de cuentas de usuario diferenciadas por roles (médico, paciente, secretaria) con permisos específicos para cada uno.
    - **Historia de Usuario (US3)**: Como paciente, quiero crear una cuenta para tener todas mis citas en un mismo lugar.
4. **Agendamiento externo con aprobación**
    - **Descripción**: El sistema permitirá a los pacientes solicitar citas desde la plataforma, pero estas quedarán en estado pendiente hasta que una secretaria las apruebe o rechace.
    - **Historia de Usuario (US11)**: Como paciente, quiero observar los horarios disponibles de un doctor para agendar sin necesidad de llamadas innecesarias.
5. **Gestión de listas de espera**
    - **Descripción**: El sistema debe permitir que los pacientes se inscriban en una lista de espera para horarios que ya están ocupados.
    - **<u>Derivada del feedback del usuario:</u>** Como paciente, quiero entrar a una lista de espera para un horario deseado y recibir una notificación si este se libera.
6. **Sistema de citas de respaldo (Fallback)**

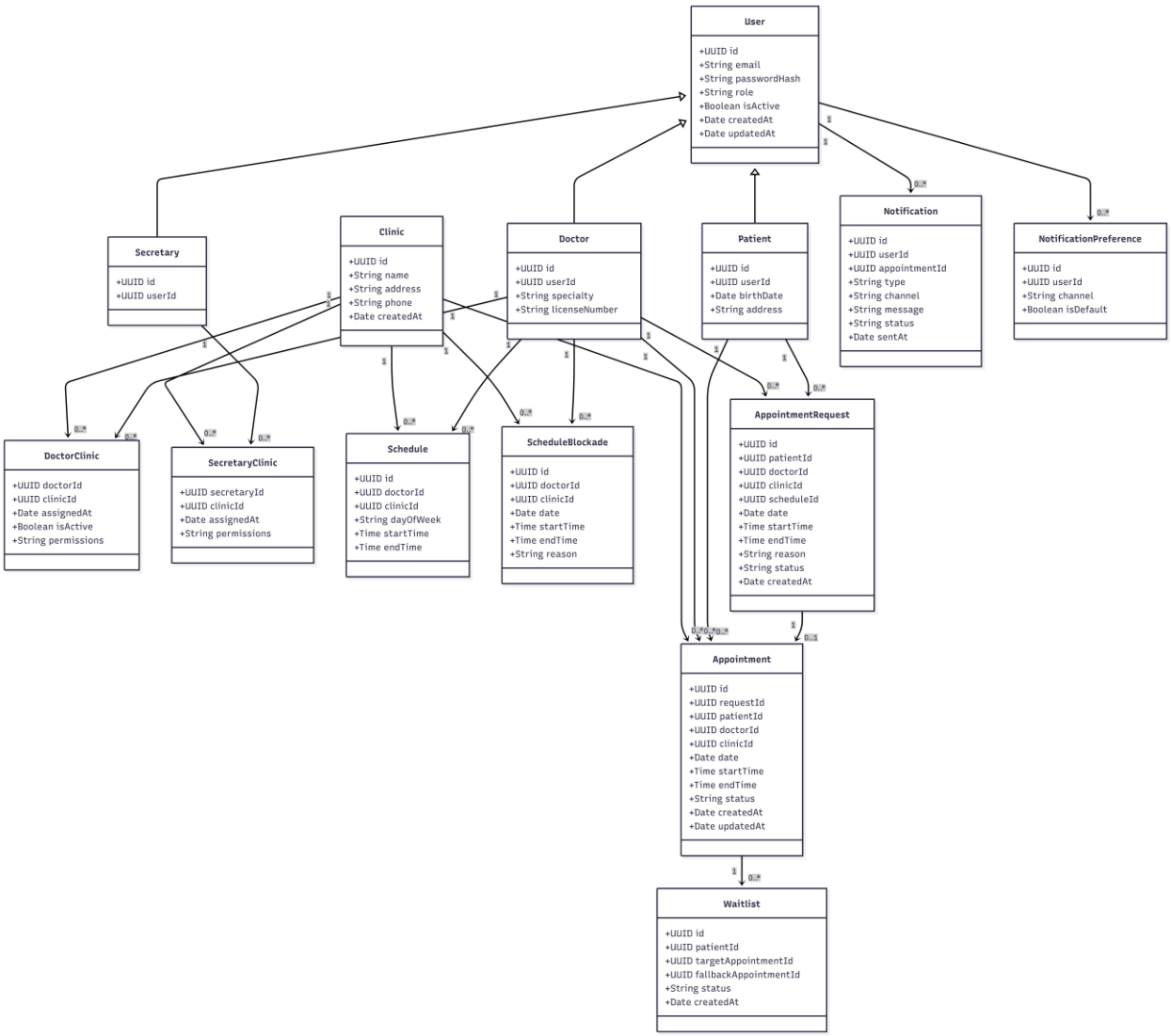
- **Descripción**: El sistema permitirá programar una cita de respaldo al entrar en una lista de espera; si el espacio de la lista de espera se libera y es aceptado, la cita de respaldo se cancelará automáticamente.
- **Derivada del feedback de secretaria:** Como secretaria, quiero que los pacientes tengan una cita de respaldo vinculada a su espera para evitar "citas fantasma" y asegurar la atención del paciente.

7. **Notificaciones inteligentes multicanal**
    - **Descripción**: El sistema enviará notificaciones automáticas por el método preferido del usuario (SMS, WhatsApp, Email) ante cualquier cambio, aprobación o liberación de espacio.
    - **Historia de Usuario (US8)**: Como paciente, quiero recibir notificaciones automáticas cuando mi cita sea cancelada o reprogramada para reorganizar mi agenda oportunamente.
8. **Bloqueo de agenda para procedimientos**
    - **Descripción**: El sistema debe permitir a los médicos bloquear espacios de tiempo específicos para cirugías, imprevistos o procedimientos que no son consultas estándar.
    - **Historia de Usuario (US9)**: Como doctor, quiero poder bloquear y modificar horarios según procedimientos o imprevistos para mantener mi agenda actualizada.
9. **Visualización de actualizaciones en tiempo real**
    - **Descripción**: El sistema debe actualizar la disponibilidad y el calendario de forma casi instantánea para todos los usuarios conectados cuando ocurra un cambio.
    - **Historia de Usuario (US7)**: Como doctor, quiero revisar actualizaciones de mi horario en tiempo real para estar siempre enterado de mis actividades.
10. **Registro simplificado de pacientes**
    - **Descripción**: El sistema almacenará la información básica del paciente (ya no expediente completo, solo nombre, teléfono/correo, etc...) de forma permanente para agilizar futuros agendamientos, eliminando el llenado de formularios repetitivos.
    - **Historia de Usuario (US6)**: Como paciente, quiero un sistema de registro simplificado que guarde mi información para ahorrar tiempo en cada visita.

Clases preliminares

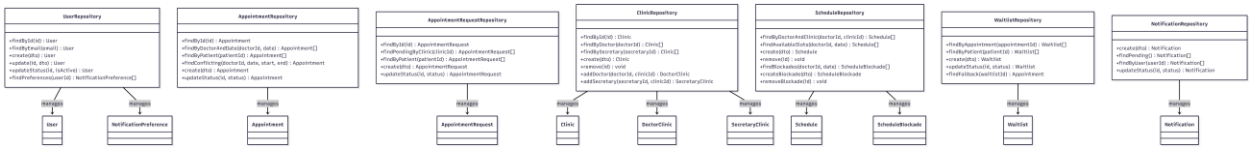
Diagrama de clases

Figura 5. Diagrama de clases para la capa Model.



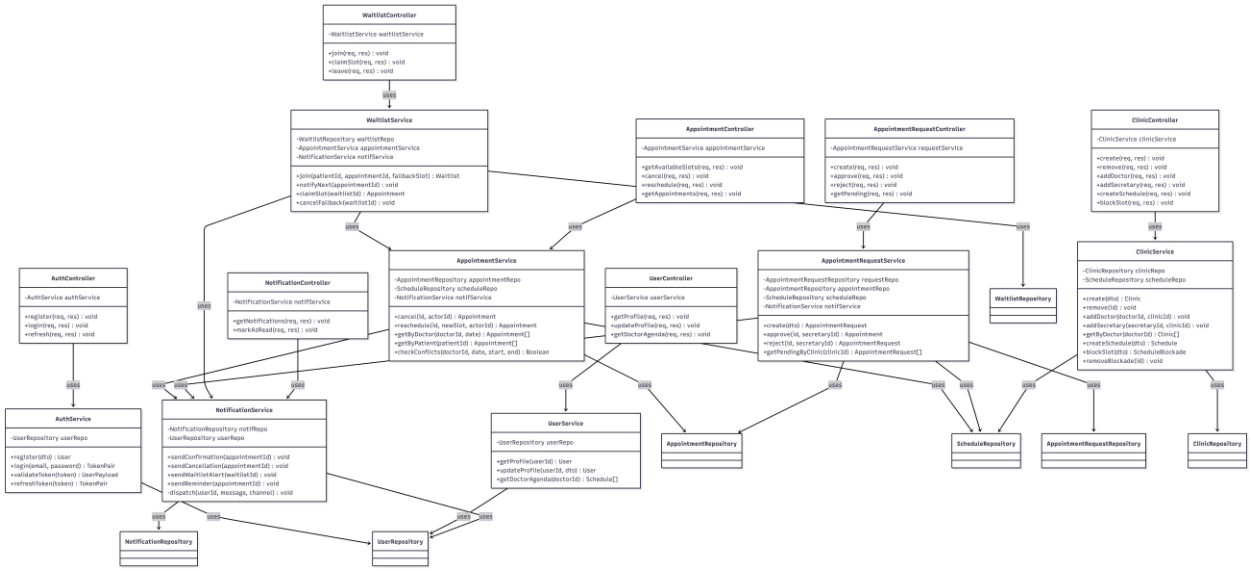
Enlace

Figura 6. Diagrama de clases para la capa de Repositories.



Enlace

Figura 7. Diagrama de clases para las capas de Services y Controllers.



Enlace

Descripción de las clases

Como se explicará en la sección de selección de tecnología, se usará una arquitectura por capas, a continuación se detallan las clases para las diferentes capas:

Model:

- User: Clase base de la que heredan los demás tipos de usuario. Centraliza la autenticación.
- Patient: (extiende User) Representa al paciente. Guarda información básica de contacto que persiste entre citas.

- Doctor: (extiende User) Representa al médico. Contiene su especialidad y la lista de clínicas donde trabaja.
- Secretary: (extiende User) Representa al personal administrativo. Está asociada a una o más clínicas con permisos específicos.
- DoctorClinic: Relación entre un médico y una clínica. Existe porque un médico puede trabajar en varias clínicas y una clínica puede tener varios doctores, y cada relación puede tener reglas y permisos propios.
- SecretaryClinic: Asignación de secretarias a clínicas. Existe porque una clínica puede tener varias secretarias y una secretaria puede trabajar para varias clínicas.
- Clinic: Representa una sede médica. Puede tener múltiples doctores y secretarias.
- Schedule: Define los bloques de disponibilidad de un doctor en una clínica específica. Es la fuente de la verdad sobre cuándo puede haber citas.
- ScheduleBlockade: Representa los bloqueos excepcionales del médico. Bloquean el horario pero no son citas necesariamente.
- Appointment: Representa una cita concreta entre paciente y doctor en una clínica.
- AppointmentRequest: Representa una solicitud de cita iniciada por paciente antes de ser aprobada.
- Waitlist: Maneja la lista de espera para un horario ocupado. Se vincula a una Appointment de respaldo.
- Notification: Registro de cada aviso enviado. Permite auditar y gestionar el envío multicanal.
- NotificationPreference: Representa el canal de preferencia de notificaciones de un usuario.

<u>Repositories:</u>

Cada repositorio encapsula todas las queries a la base de datos de su entidad. Los servicios nunca hablan directamente con el ORM o con la base de datos, solo con el repositorio.

- UserRepository
- AppointmentRepository
- AppointmentRequestRepository
- ClinicRepository
- ScheduleRepository
- WaitlistRepository
- NotificationRepository

<u>Services:</u>

- AuthService: Maneja registro, login, generación y verificación de tokens JWT. Valida credenciales y determina permisos según el rol del usuario.

- UserService: Gestiona perfiles de usuarios según su rol. Diferencia entre las operaciones permitidas para pacientes, doctores y secretarias.
- AppointmentService: Coordina la creación de citas verificando disponibilidad en tiempo real, aprobación por secretaria, y el manejo de conflictos entre clínicas del mismo doctor.
- AppointmentRequestService: Maneja todas las solicitudes de citas y su aprobación o rechazo.
- WaitlistService: Gestiona la lógica de la lista de espera y las citas de respaldo. Cuando se cancela una cita, notifica al primero en la lista y cancela automáticamente su cita de respaldo si acepta.
- ClinicService: Administra clínicas, su relación con doctores y secretarias, y los horarios dentro de cada una.
- NotificationService: Abstrae el envío de notificaciones multicanal. Internamente delega al canal correcto (email, SMS, WhatsApp) según la preferencia del usuario.

<u>Controllers:</u>

Cada controller recibe el request HTTP, extrae y valida los parámetros, llama al servicio correspondiente y retorna el response

- AuthController
- UserController
- AppointmentController
- AppointmentRequestController
- ClinicController
- WaitlistController
- NotificationController

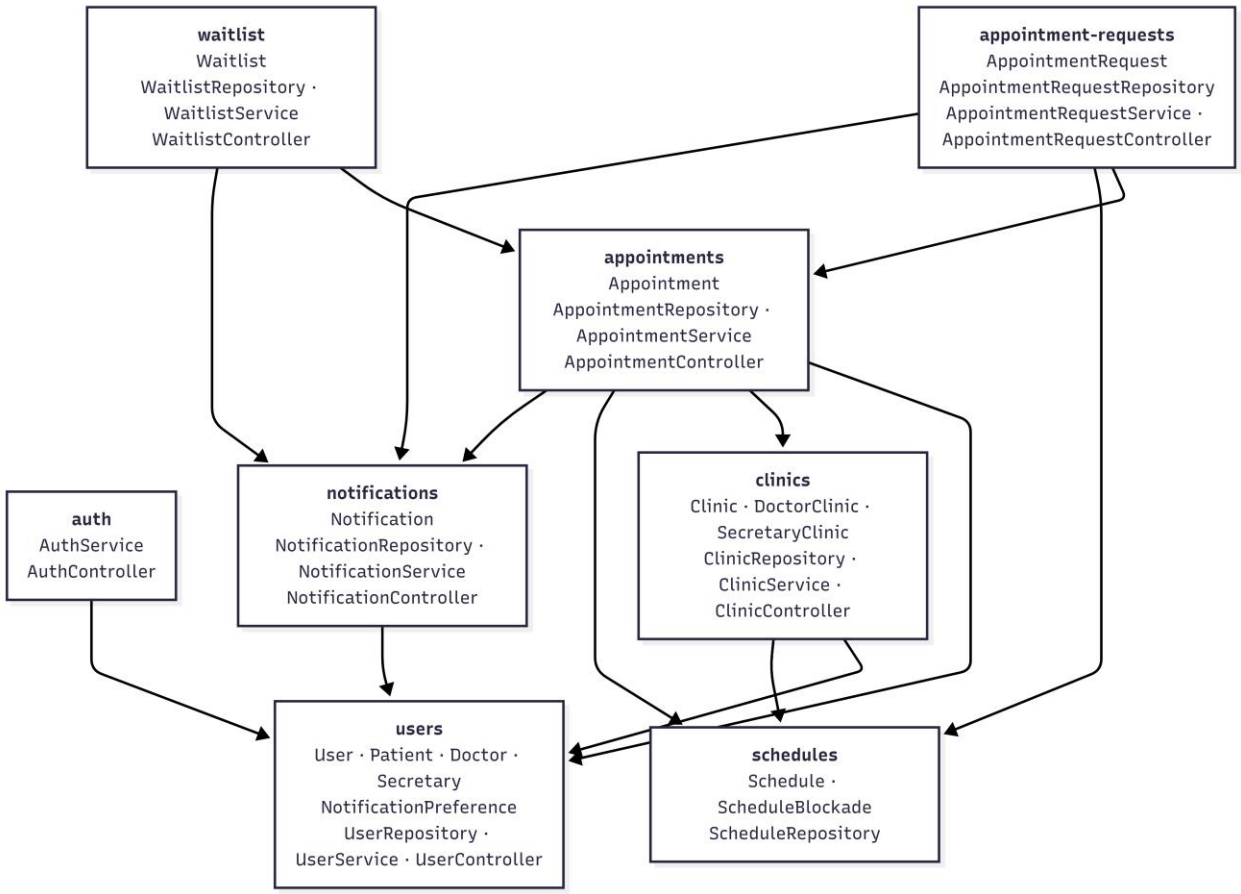
<u>Routes</u>

No son clases, sino archivos con las rutas con middlewares de autenticación y autorización por rol antes de llamar al controller.

- authRoutes
- userRoutes
- appointmentRoutes
- appointmentRequestRoutes
- clinicRoutes
- waitlistRoutes
- notificationRoutes

Diagrama de paquetes

Figura 8. Diagrama de paquetes.



Enlace

Descripción de los paquetes

Auth:

Responsable exclusivamente de la autenticación. Verifica credenciales, genera tokens JWT y los valida en cada request. Depende de users para consultar si el usuario existe y cuál es su rol, pero no conoce nada del resto del sistema. Es el paquete con menor acoplamiento de todos. Contiene AuthService y AuthController. No tiene model propio porque usa User del paquete users.

Users:

Define las entidades base del sistema. Es el paquete del que más dependen los demás porque User y sus subtipos (Patient, Doctor, Secretary) son referenciados en casi todas las operaciones.

Contiene User, Patient, Doctor, Secretary, NotificationPreference, UserRepository, UserService y UserController.

Clinics:

Gestiona las sedes médicas y la asignación de doctores y secretarias a ellas. Depende de users para validar que los actores existen, y de schedules para crear y administrar los horarios de cada clínica. Contiene Clinic, DoctorClinic, SecretaryClinic, ClinicRepository, ClinicService y ClinicController.

Schedules:

Define cuándo puede atender un doctor en cada clínica. Contiene los bloques de disponibilidad semanales y los bloqueos puntuales por cirugías o imprevistos. No tiene controller propio porque sus operaciones se exponen a través de ClinicController. Contiene Schedule, ScheduleBlockade, ScheduleRepository, y la lógica de disponibilidad que vive en ClinicService. No tiene controller propio porque sus endpoints se exponen a través de ClinicController.

Appointments:

Gestiona el ciclo de vida de las citas ya aprobadas: cancelación, reagendamiento, consulta de agenda y detección de conflictos de horario. Contiene Appointment, AppointmentRepository, AppointmentService y AppointmentController.

Appointment-requests:

Maneja el flujo previo a la cita: el paciente solicita, la secretaria aprueba o rechaza, y si aprueba se crea la Appointment en el paquete appointments. Contiene AppointmentRequest, AppointmentRequestRepository, AppointmentRequestService y AppointmentRequestController.

Waitlists:

Gestiona la cola de espera para horarios ocupados y las citas de respaldo vinculadas. Cuando se libera un slot, notifica al siguiente en la lista y cancela automáticamente su cita de respaldo si decide tomar el nuevo horario. Contiene Waitlist, WaitlistRepository, WaitlistService y WaitlistController.

Notifications:

Servicio transversal que abstrae el envío de mensajes por email, SMS o WhatsApp según la preferencia de cada usuario. Es llamado por appointments, appointment-requests y waitlist Contiene Notification, NotificationPreference (compartida con users), NotificationRepository, NotificationService y NotificationController.

Persistencia de datos

Selección de tipo de base de datos

Base de datos relacional (SQL):

Es el modelo más maduro y ampliamente utilizado. Organiza los datos en tablas con filas y columnas, donde cada tabla representa una entidad y las relaciones entre entidades se establecen mediante claves foráneas. Utiliza SQL como lenguaje estándar de consulta y garantiza las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), lo que significa que cada transacción se completa en su totalidad o no se completa en absoluto.

Sus fortalezas principales son la integridad referencial, el soporte nativo para joins complejos entre múltiples tablas, y la consistencia estricta de los datos. Sus limitaciones aparecen cuando los datos son muy poco estructurados o cuando se necesita escalar horizontalmente de forma masiva, ya que el esquema rígido dificulta cambios frecuentes en la estructura.

Base de datos documental (NoSQL):

Almacena los datos como documentos en formato JSON o BSON, donde cada documento es una unidad autocontenida que puede tener una estructura diferente a los demás. No requiere un esquema fijo, lo que permite agregar campos nuevos sin migraciones. Los representantes más conocidos son MongoDB y CouchDB.

Sus fortalezas son la flexibilidad del esquema, la velocidad de lectura cuando los datos relacionados se guardan juntos en un mismo documento, y la facilidad de escalar horizontalmente. Sus limitaciones aparecen cuando hay relaciones complejas entre entidades, ya que los joins no son nativos y deben resolverse en el código de la aplicación, lo que transfiere la responsabilidad de la integridad de los datos al developer.

Base de datos en grafo

Modela los datos como nodos y aristas, donde los nodos son las entidades y las aristas son las relaciones entre ellas. Son excepcionales cuando las relaciones entre datos son el centro del problema, como redes sociales, sistemas de recomendación o detección de fraude. Neo4j es el ejemplo más conocido. Sus limitaciones aparecen cuando el dominio no es inherentemente relacional en el sentido de grafos, ya que la ventaja de rendimiento desaparece y la complejidad operacional no se justifica.

Selección para MediNet:

El tipo de base de datos seleccionado es relacional, justificado directamente por las características del dominio.

El sistema tiene catorce entidades con relaciones bien definidas y obligatorias entre sí. Una cita no existe sin un paciente, un doctor, una clínica y un schedule. Una lista de espera apunta a dos citas distintas. Estas relaciones son la esencia del negocio, y el modelo relacional las expresa de forma natural mediante claves foráneas, mientras que otros modelos obligarían a resolverlas en el código de la aplicación.

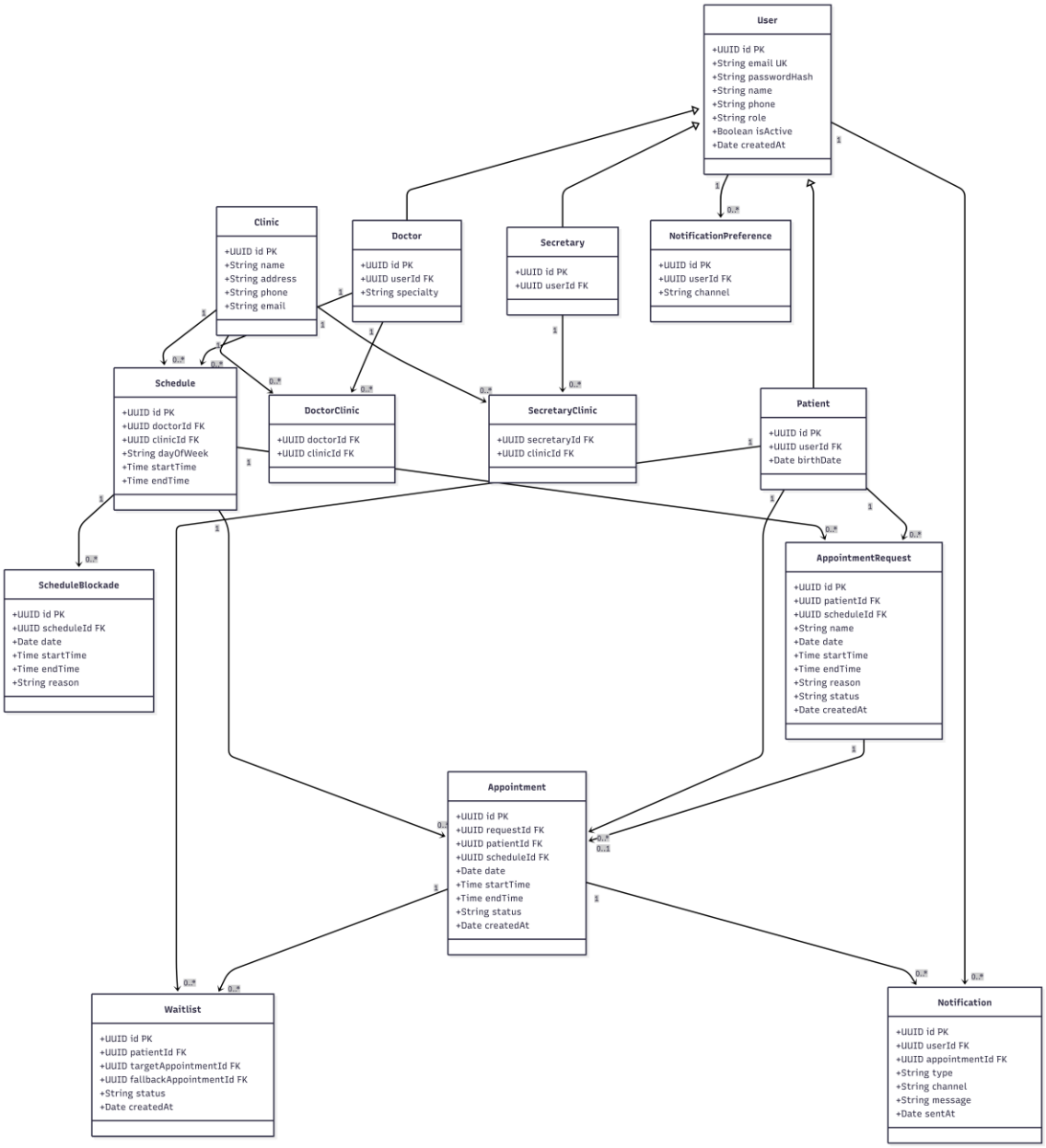
La integridad de los datos es crítica en un sistema de salud. Las transacciones ACID garantizan que el motor rechace activamente datos inconsistentes, como dos citas en el mismo slot o una cita apuntando a un doctor inexistente. Ningún otro tipo de base de datos ofrece estas garantías de forma nativa.

Las consultas más importantes del sistema, como detectar conflictos de horario o construir la agenda completa de un médico, requieren cruzar múltiples tablas simultáneamente. SQL resuelve esto en una sola consulta optimizada, mientras que un modelo documental requeriría múltiples roundtrips o datos duplicados difíciles de mantener.

Finalmente, el volumen estimado del sistema no supera los 50 GB en varios años de operación y 400 usuarios concurrentes, una carga que una base de datos relacional maneja con comodidad en un único servidor, sin necesidad de la complejidad operacional de soluciones distribuidas.

Diagrama de clases persistentes

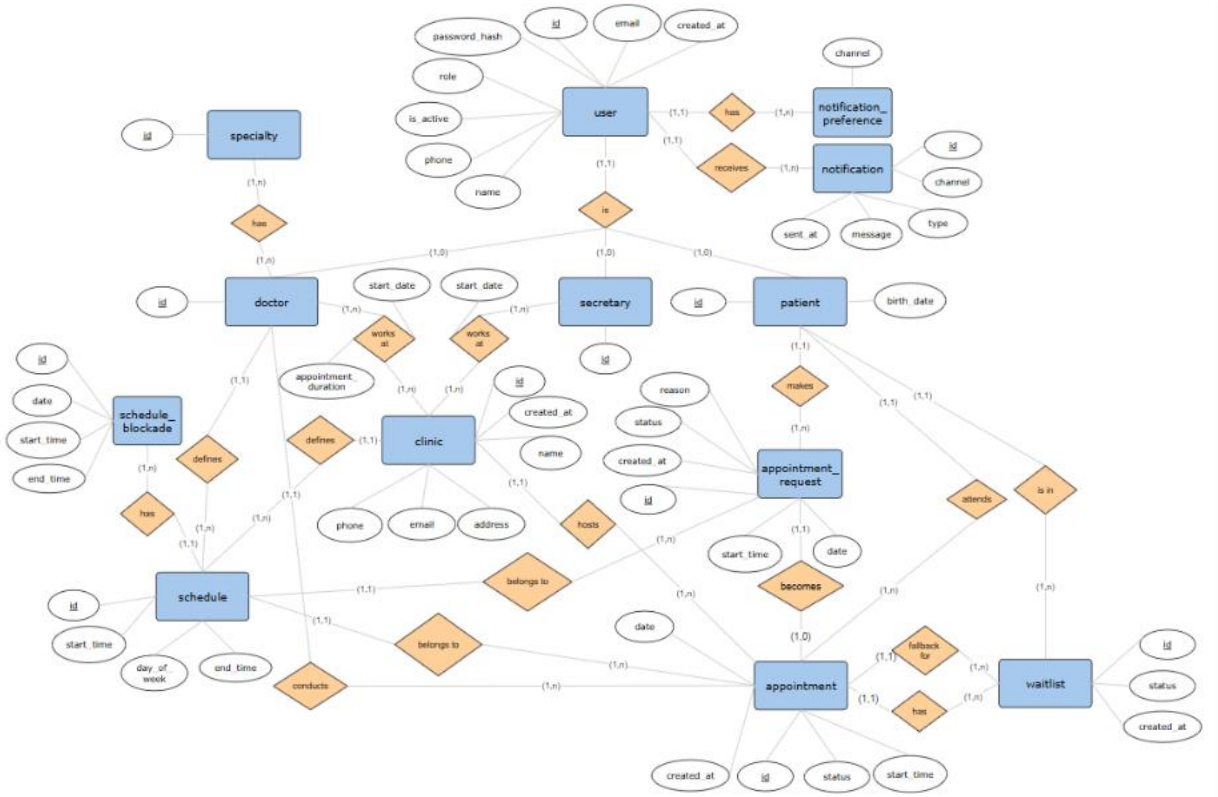
Figura 9. Diagrama de clases persistentes.



Enlace

Diagrama entidad-relación

Figura 10. Diagrama entidad-relación.



Diseño

Selección de tecnología

Estimaciones de dimensiones del sistema

Cantidad de usuarios totales:

Según la elaboración de Worldometer de los datos más recientes de las Naciones Unidas, Guatemala posee una población de aproximadamente 18.8 millones de habitantes (Worldometer, 2026). Con una tasa de médicos de 1.2 por cada 1,000 personas (Valle, 2020), se estima la presencia de aproximadamente 22,560 médicos colegiados en el país. Si MediNet aspira a capturar inicialmente un segmento del sector privado, el cual atiende a una porción significativa de la población que no cuenta con cobertura de seguridad social (solo el 7.87% está cubierto por el IGSS (DAE, 2022)), el volumen de transacciones potenciales es considerable.

Tomando estos datos en cuenta, se puede hacer una estimación inicial que la captura de 100 clínicas médicas y sus respectivos 350 médicos aproximados (0.0155% del total de médicos) representarían un total de pacientes de alrededor de 291,000 (0.0155% de la población guatemalteca). Sin embargo, sabiendo que solo alrededor del 70% de la población guatemalteca tiene acceso a servicios de salud (Becerril-Montekio & López-Dávila, 2011) y asumiendo que el 33.8% de los pacientes son menores de edad (INE, 2023) y que el 4% son mayores de 70 (INE, 2018), y que ellos no programan sus propias citas médicas, podemos obtener un gran total aproximado de **126,000 pacientes** en el país.

Tiempo promedio de respuesta:

La latencia es un factor crítico para la experiencia del usuario en sistemas de tiempo real. Los estándares de la industria indican que una latencia inferior a 100ms es percibida como instantánea por el usuario (Essential Designs Team, 2025). En el contexto de MediNet, la latencia se desglosa en:

- Latencia de Red: El tiempo que tarda el paquete en viajar del cliente al servidor. En América Latina, esto depende de la proximidad a los centros de datos.
- Latencia de Procesamiento: El tiempo que el backend tarda en ejecutar la lógica y consultar la base de datos. Con un backend optimizado y una base de datos indexada, esto debe mantenerse <50ms.

- Latencia de Sincronización: El retraso en propagar un cambio a otros clientes conectados, el cual debe ser minimizado mediante el uso de protocolos como WebSockets o Server-Sent Events (SSE).

Al no tratarse de una aplicación crítica en donde los tiempos de respuesta deban ser inmediatos, un promedio de **200ms** es totalmente aceptable.

Cantidad de usuarios simultáneos:

Para calcular la carga de usuarios concurrentes que el sistema debe soportar en un escenario de crecimiento, se aplica la fórmula de tráfico basada en sesiones (Cardinali, 2023):

$$C = \frac{n \times L}{T}$$

Donde:

- n es el número de visitas o sesiones esperadas en un periodo determinado.
- L es la duración promedio de la sesión en minutos.
- T es la duración del periodo total en minutos.

Si consideramos una meta de 100 clínicas registradas, cada una atendiendo un promedio de 20 pacientes al día, el sistema procesará 2,000 sesiones diarias solo por parte de pacientes, con un tiempo de interacción para agendar de 3 minutos. Además, si el personal administrativo (2 secretarias por clínica) mantiene sesiones prolongadas de cerca de 8 horas para gestión, y los médicos también revisan sus horarios a diario por 3 minutos, el sistema debe estar preparado para picos de concurrencia superiores a los:

$$C = \frac{2200 \times 3}{480} = 13.75 \text{ pacientes y doctores}$$

y contando el personal administrativo conectado por sesiones largas, el sistema debe manejar cantidades superiores a los:

$$13.75 + 350 = 363.75 \text{ usuarios}$$

en horarios de alta demanda. Sin embargo, para evitar complicaciones inesperadas, el sistema deberá soportar picos de por lo menos **400 usuarios.**

Tamaño en disco:

Un registro estándar en una base de datos relacional para una cita médica incluye típicamente: ID de cita, ID de paciente, ID de médico, ID de clínica, fecha, hora, estado y una breve nota de motivo. Aproximando para los tipos comunes de datos que guardan esta información (Int64, Varchar(255), Date, Text), un registro en la base de datos tendrá un tamaño aproximado de 340 bytes.

Para un volumen de 10 millones de citas almacenadas (equivalente a varios años de operación a gran escala), el espacio en disco requerido para los datos brutos sería de aproximadamente 3.4 GB. Incluyendo otras tablas similares (Usuarios, Clínicas, Médicos, etc...), índices, logs de auditoría y copias de seguridad, el almacenamiento total para este volumen de registros no excedería los **40-50 GB**, lo que permite que la base de datos sea altamente eficiente y que gran parte de los índices residan en la memoria RAM para optimizar la latencia.

Disponibilidad de la información:

Los estándares de la industria para pequeñas empresas en cuanto a disponibilidad de la información rondan el 99 a 99.9% (Bubobot, 2025). Al MediNet ser una empresa nueva no-crítica, sin ingresos estables ni trabajadores de tiempo completo, una **disponibilidad del 99%** es una aproximación válida.

Confiabilidad o tolerancia a fallos:

Asumiendo 99% de disponibilidad, el sistema fallaría por unas 88 horas al año. Asumiendo también que el tiempo de reparación es de 1 hora, esto significa que el sistema fallaría por 1 hora cada 4.1 días o 99 horas aproximadamente. Modelando esto como una tasa de fallos constante λ :

$$R(t) = e^{-\lambda t}$$

Donde:

- $R(t)$ es la confiabilidad en el tiempo t.
- λ es la tasa de fallos por hora.
- t es el tiempo de operación.

Quedaría:

$$R(t) = e^{-\left(\frac{1}{99}\right)(24)} = 0.785$$

Esto significa que hay **78.5% de probabilidad de que el servidor no falle en un día**, o 21.5% de probabilidad que falle en un día.

Tecnología por utilizar

Framework y Lenguaje de programación:

1. FastAPI (Python)

FastAPI es un framework web de Python moderno y de alto rendimiento diseñado para crear APIs con documentación interactiva automática y seguridad de tipos. Ofrece velocidad excepcional para procesamiento de datos en tiempo real y aplicaciones web flexibles.

   a. Ventajas
      i. Ciclo de desarrollo rápido.
      ii. Gestión de miles de conexiones simultáneas de forma eficiente.
      iii. Documentación automática de API con Swagger UI y ReDoc.
      iv. La validación de solicitudes integrada mediante modelos Pydantic detecta datos mal formados antes de que lleguen a la parte de business-logic.
      v. Boilerplate mínimo.
      vi. El sistema de inyección de dependencias simplifica las pruebas.
   b. Desventajas
      i. Ecosistema más pequeño en comparación con otros frameworks de Python como Flask o Django.
      ii. Framework relativamente joven por lo que cambios importantes entre versiones puede requerir un esfuerzo de migración.
      iii. Funcionalidades integradas limitadas para requisitos empresariales complejos.
   c. Métricas
      i. 500,000 desarrolladores.
      ii. Bundle size de 50-80MB para una imagen de Docker con Python slim y dependencias.
      iii. Uso de memoria de 40-80MB por worker.
2. Node.js, Fastify (JavaScript)

Node.js es un entorno de ejecución de JavaScript basado en el motor V8 de Chrome que permite la ejecución de JavaScript del lado del servidor, lo que lo hace ideal para crear sistemas backend flexibles. Su modelo de I/O no bloqueante es fundamental para el desarrollo backend, ya que permite crear aplicaciones de alto rendimiento que gestionan miles de conexiones concurrentes de forma eficiente. Su arquitectura orientada a eventos destaca en aplicaciones en tiempo real, microservicios, APIs RESTful y servicios con gran volumen de datos que requieren un procesamiento rápido y escalabilidad horizontal.

   a. Ventajas
      i. 50,000 requests por segundo para operaciones I/O.

ii. El uso de JavaScript en todo el stack permite a los desarrolladores de backend trabajar en el frontend cuando sea necesario.
iii. El ecosistema masivo de NPM proporciona más de 2 millones de paquetes que cubren autenticación, controladores de bases de datos, marcos de API y utilidades.
iv. El manejo nativo de JSON hace que el desarrollo de API RESTful sea sencillo, ya que los objetos JavaScript se asignan directamente a JSON, eliminando la serialización.
v. Fuerte apoyo de la comunidad y amplia documentación.
vi. La arquitectura basada en eventos se adapta naturalmente al manejo de webhooks, el procesamiento de colas de mensajes y los patrones de ejecución de tareas. asíncronas.

b. Desventajas
i. El bucle de eventos de un solo hilo puede convertirse en un cuello de botella para operaciones que consumen muchos recursos de la CPU.
ii. El “callback hell” y la complejidad asincrónica pueden generar un código difícil de mantener a pesar de las promesas y async/await, especialmente para los desarrolladores junior.
iii. El tipado débil en JavaScript puro aumenta los errores en tiempo de ejecución y dificulta la refactorización segura de grandes bases de código.
iv. Las fugas de memoria son más fáciles de introducir debido a los "closures” y “event listeners”, lo que requiere una supervisión y un análisis cuidadosos en los entornos de producción.
v. Menos adecuado para cargas de trabajo computacionales pesadas o procesamiento intensivo de datos en comparación con Java, Go o Python.

c. Métricas
i. 20 millones de desarrolladores.
ii. Bundle Size de 100 MB a 1 GB o más ya que la carpeta node_modules puede alcanzar un tamaño considerable.
iii. Uso de memoria de 100-500MB para aplicaciones típicas.

3. Go

Go es un lenguaje de programación compilado y de tipado estático, diseñado por Google para crear sistemas backend flexibles y de alto rendimiento. Destaca en la gestión de operaciones concurrentes, lo que lo hace ideal para arquitecturas de microservicios, API y sistemas distribuidos. Su rápida compilación, eficiente gestión de memoria y primitivas de concurrencia integradas permiten a los desarrolladores crear servicios robustos que gestionan millones de solicitudes con una mínima sobrecarga de recursos, incluso sin un framework de desarrollo.

a. Ventajas
i. 2-10 veces más rápido para tareas donde el cuello de botella es el procesador.

ii. Un modelo de concurrencia excepcional con goroutines y canales permite gestionar miles de conexiones simultáneas de forma eficiente.
iii. Las bibliotecas de red y el servidor HTTP integrados están listos para su uso inmediato, eliminando la dependencia de frameworks pesados.
iv. La tipificación estática fuerte detecta errores en tiempo de compilación, lo que reduce los errores de ejecución en los backends de producción.
v. El despliegue de un único binario sin dependencias en tiempo de ejecución simplifica la contenerización y los procesos de despliegue.
vi. Excelente cobertura de bibliotecas estándar para tareas comunes de backend.

b. Desventajas
   i. Más código repetitivo y aserciones de tipo, lo que aumenta el tiempo de desarrollo para estructuras de datos complejas y reduce la reutilización del código.
   ii. El manejo de errores mediante verificación explícita crea un código verboso con patrones repetitivos if-err-nil.
   iii. Falta de características tradicionales orientadas a objetos como la herencia y la sobrecarga de métodos.
   iv. La gestión de dependencias y el control de versiones han sido históricamente problemáticos.

c. Métricas
   i. 3-4 millones de desarrolladores.
   ii. Bundle size de 5 a 15 MB. Solo un archivo binario pequeño.
   iii. Uso de memoria de 10-50MB para aplicaciones básicas.

(Index, 2026)

<u>Frontend:</u>

1. Flutter

Flutter es un software development framework open-source para crear aplicaciones iOS, Android y Web desde un solo código en Dart. El código Dart se compila previamente en código máquina nativo mediante el compilador Flutter, lo que permite que las aplicaciones alcancen un rendimiento excepcionalmente alto. Una característica destacada de Flutter es su funcionalidad de recarga instantánea increíblemente rápida, que permite a los desarrolladores experimentar rápidamente y visualizar los cambios en la interfaz de usuario casi al instante.

a. Ventajas
   i. Código fuente único de Dart.
   ii. El “hot reload” con estado permite actualizaciones visuales instantáneas.
   iii. Excelente rendimiento.
   iv. Amplio catálogo de materiales personalizables.

    v. Dart es un lenguaje de programación expresivo y fácil de aprender.
  b. Desventajas
    i. Ecosistema de bibliotecas y herramientas más pequeño que las alternativas más viejas.
    ii. Centrado principalmente en el desarrollo de aplicaciones móviles, es menos adecuado para la web.
    iii. Es probable que se produzcan cambios arquitectónicos y actualizaciones importantes a medida que el marco de trabajo madure.
2. React

React es una biblioteca de JavaScript de código abierto desarrollada por Facebook para el desarrollo de interfaces de usuario interactivas y front-ends de aplicaciones web. React se basa en HTML, CSS y JavaScript para proporcionar un modelo de componentes declarativo. En este modelo, las interfaces de usuario se dividen en partes encapsuladas y reutilizables llamadas componentes. Los componentes utilizan un lenguaje de plantillas llamado JSX, similar a HTML, para renderizar contenido, pueden tener un estado local y se actualizan dinámicamente en respuesta a cambios en los datos. React promueve el acoplamiento flexible entre componentes y fomenta el uso de interfaces sencillas y reutilizables en lugar de implementaciones concretas.

  a. Ventajas
    i. Promueve la reutilización mediante componentes encapsulados.
    ii. Los componentes ofrecen separación de responsabilidades
    iii. El DOM virtual proporciona excelentes mejoras de rendimiento.
    iv. El flujo de datos unidireccional simplifica la lógica de manejo de datos.
    v. Hay un amplio ecosistema de código abierto de bibliotecas de componentes reutilizables de React
    vi. La creciente comunidad de desarrolladores proporciona abundantes recursos de soporte.
  b. Desventajas
    i. Se necesitan muchas bibliotecas auxiliares para aplicaciones complejas.
    ii. Complejidad del código derivada de JSX y árboles de componentes anidados
    iii. Curva de aprendizaje pronunciada para dominar el funcionamiento interno, como el estado y el ciclo de vida.
    iv. La evolución rápida implica cambios en la API que rompen la compatibilidad, lo cual no es infrecuente.
3. Angular

Angular es un framework de desarrollo de aplicaciones web mantenido por Google. Las aplicaciones Angular se escriben en TypeScript, una versión mejorada de JavaScript que incorpora tipado opcional y otras mejoras orientadas a aplicaciones de gran escala. El framework utiliza una arquitectura MVC para separar las funcionalidades de una aplicación en componentes distintos: Modelos, Vistas y Controladores. Angular también ofrece un excelente soporte para

formularios y su validación, lo que facilita la creación de flujos complejos de entrada de datos. El framework simplifica el enrutamiento y la navegación, así como la comunicación entre componentes.

a. Ventajas
    i. El framework integra una amplia gama de funciones web.
    ii. La arquitectura MVC permite la separación de responsabilidades.
    iii. Amplio conjunto de herramientas que incluye conjuntos de pruebas y análisis de código.
    iv. Gran énfasis en la reutilización y el mantenimiento.
    v. La gran comunidad ofrece muchas bibliotecas reutilizables.
b. Desventajas
    i. Curva de aprendizaje pronunciada debido a la gran superficie de API.
    ii. Aplicaciones más pesadas debido a una gran base de código del framework.
    iii. Las actualizaciones entre versiones pueden requerir una importante revisión.
    iv. Los tiempos de arranque y renderizado más prolongados son consecuencia de la complejidad.

(Simplilearn, 2025)

<u>Arquitectura:</u>

1. Model View Controller (MVC)

Patrón basado en la idea principal que la interfaz del usuario no tiene que manejar de forma directa la lógica ni la base de datos, por lo que separa un sistema en tres componentes para poder dividir responsabilidades efectivamente entre la interfaz del programa, la lógica y el manejo de los datos. El Model, se encarga de los datos y las interacciones con la base de datos, el View de la interfaz que el usuario puede ver y el Controller funciona como un intermediario entre el Model y el View.

a. Ventajas
    i. Es una arquitectura bien documentada, ya que es una de las más utilizadas.
    ii. Facilita el desarrollo paralelo, por lo que distintos integrantes del grupo pueden trabajar en distintas partes de sistema.
b. Desventajas
    i. No es el mejor para manejar sistemas altamente reactivos, como el que se está planteando, ya que este tiene notificaciones, listas de esperas y eventos de calendario.
    ii. Escalabilidad limitada, ya que si el sistema crece mucho la lógica es más difícil de mantener y manejar muchos controllers llega a ser complicado.

2. Monolito Modular

Arquitectura que define un sistema como una unidad de despliegue única, pero internamente está dividida en módulos independientes.

a. Ventajas
    i. Fácil desarrollo, gracias a que se tiene un solo backend, un solo despliegue y una sola base de datos.
    ii. Infraestructura poco costosa, porque no se necesitan message brokers u orquestadores.
b. Desventajas
    i. La escalabilidad es limitada, ya que, si una parte del sistema necesita más recursos, se tiene que escalar el sistema completo.
    ii. Existe el risgo de que los modulos del programa dependan mucho entre ellos, si se hace un mal diseño.
    iii. Un cambio pequeño en el sistema requiere que se despliegue todo de nuevo.

3. Event Driven

Arquitectura asíncrona donde los distintos componentes del sistema se comunican entre ellos produciendo o consumiendo eventos, siendo estos cambios significativos en el estado como un usuario creando una nueva cita. Sus componentes principales son los productores de eventos que detectan cambios y los publican al Event Broker, el cual se encarga de almacenar y enrutar los eventos hacia los consumidores, los cuales reaccionan a los eventos que reciban.

a. Ventajas
    i. Bajo acoplamiento entre componentes, ya que los sistemas no se llaman directamente, si no que solamente reaccionan a eventos.
    ii. Resiliencia con los datos, ya que en caso de que un evento falle al ser consumido, este seguirá almacenado en el Event Broker para ser procesado de nuevo.
b. Desventajas
    i. Debugging difícil a causa de la separación e independencia que existe entre los servicios del sistema.
    ii. Los sistemas basados en eventos no son instantáneamente consistentes, los datos pueden tardar en ser actualizados en todo el sistema

4. Arquitectura en Capas

La arquitectura en capas consta en dividir la aplicación en capas, con la intención de que cada capa tenga un rol muy definido, como podría ser, una capa de presentación (UI), una capa de reglas de negocio (servicios) y una capa de acceso a datos (DAO), sin embargo, este estilo arquitectónico no define cuantas capas debe de tener la aplicación, sino más bien, se centra en la separación de la aplicación en diferentes preocupaciones bien definidas.

a. Ventajas
    i. Permite la separación de preocupaciones (SoC), ya que cada capa tiene una sola responsabilidad.

    ii. Este estilo arquitectónico es especialmente fácil de implementar, además de que es muy conocido y una gran mayoría de las aplicaciones la utilizan.
    iii. Debido a que la aplicación construida por capas, es posible ir probando de forma individual cada una para simplificar el testeo.
  b. Desventajas
    i. Por lo general, todas las capas de la aplicación son construidas con la misma tecnología, lo que hace amarremos la comunicación con tecnologías propietarias de la plataforma utilizada.
    ii. Si una capa falla, todas las capas superiores comienzan a fallar en cascada.
5. Cliente-servidor

En una arquitectura Cliente-Servidor existe un servidor y uno o múltiples clientes que se conectan al servidor para recuperar todos los recursos necesarios para funcionar, en este sentido, el cliente solo es una capa para representar los datos y se detonan acciones para modificar el estado del servidor, mientras que el servidor es el que hace todo el trabajo pesado.

  a. Ventajas
    i. El servidor fungirá como única fuente de la verdad, lo que impide que los clientes conserven información desactualizada.
    ii. El servidor por lo general está protegido por firewall o subredes que impiden que los atacantes pueden acceder a la base de datos o los recursos sin pasar por el servidor.
    iii. El cliente es por lo general una aplicación simple que no tiene dependencias, por lo que es muy fácil de instalar.
    iv. La arquitectura cliente-servidor permite implementar la lógica de negocio de forma separada del cliente.
  b. Desventajas
    i. Una cantidad no esperada de usuarios concurrentes puede ser un problema para el servidor, quien tendrá que atender todas las peticiones de forma simultánea
    ii. Si el servidor se cae, todos los clientes quedarán totalmente inoperables.
    iii. Los protocolos más utilizados para establecer comunicación entre el cliente y el servidor suelen ser de bajo nivel, como Sockets, HTTP, RPC, etc. Lo que puede implicar un reto para los desarrolladores.
    iv. Es difícil analizar un error, pues los clientes están distribuidos en diferentes máquinas.

(ReactiveProgramming, 2021)

<u>Control de versiones:</u>

1. Git y Github

Git es un sistema de control de versiones distribuido que permite registrar, gestionar y rastrear los cambios realizados en archivos a lo largo del tiempo. Es ampliamente utilizado en el desarrollo de software para facilitar la colaboración entre múltiples desarrolladores, mantener un historial de modificaciones del código y permitir la creación de ramas para trabajar en nuevas funcionalidades sin afectar la versión principal del proyecto.

a. Ventajas
    i. A diferencia de los sistemas centralizados como SVN, Git permite que cada desarrollador mantenga una copia completa del repositorio.
    ii. Las confirmaciones, el historial y las ramificaciones funcionan sin acceso a internet.
    iii. No existe un único punto de fallo. Cualquier copia puede restaurar el proyecto.
    iv. Las capacidades de ramificación y fusión de Git se encuentran entre sus características más potentes, ofreciendo una flexibilidad que otros sistemas de control de versiones tienen dificultades para igualar.
    v. Maneja proyectos grandes e historiales complejos con facilidad, lo que hace que operaciones como confirmaciones, ramificaciones y fusiones sean más rápidas en comparación con otros sistemas de control de versiones.
    vi. Tiene flexibilidad, puede diseñarse para adaptarse a una amplia gama de flujos de trabajo, desde historiales lineales simples hasta estrategias de ramificación complejas utilizadas en grandes equipos.
    vii. Git se beneficia de una comunidad grande y activa y un rico ecosistema de herramientas, lo que facilita encontrar soporte.

(Geeksforgeeks, 2025)

<u>Selección final:</u>

Como decisión final del stack tecnológico para este proyecto se decidió implementar una arquitectura híbrida, utilizando la arquitectura Cliente-Servidor para el sistema en general y la arquitectura por capas para el backend. Se eligió Node.js con el framework Fastify para la API y backend, mientras que se eligió Flutter para la aplicación cliente y el frontend, además, se decidió usar git para control de versiones.

La decisión de usar Node.js surge de su capacidad de manejar hasta 50,000 requests por segundo para operaciones I/O, en donde es claramente superior a FastAPI y a Go. Esto es una característica extremadamente importante para este sistema en particular pues, como ya se mencionó, este utilizará Node.js como una API que recibirá y enviará información de vuelta a un cliente. Además, como se mencionó en las estimaciones del sistema, se prevee hasta 400 usuarios simultáneos interactuando con el sistema y esperando tiempos de respuesta menores a los 200 ms, por lo que es de vital importancia utilizar un framework que sea capaz de manejar

estas demandas. Finalmente, el manejo nativo de JSON hace que el desarrollo RESTful sea sencillo y la enorme comunidad y ecosistema NPM facilitan el desarrollo enormemente.

Por otro lado, la decisión de usar Flutter para el fronten surge principalmente de la capacidad de esta herramienta para generar código nativo para iOS, Android, Desktop y Web desde un código fuente único, lo que es muy importante para una aplicación en donde una de las prioridades es la portabilidad para que los usuarios puedan estar siempre al tanto de sus horarios. Esta portabilidad sería mucho más difícil de conseguir usando Angular o React ya que estas son herramientas pensadas principalmente para la Web. Además, el “hot reload” facilita el desarrollo de aplicaciones funcionales y atractivas sin perder el tiempo en recompilaciones.

Por último, se decidió usar git y GitHub para el control de versiones por su superioridad evidente a otros controladores y su popularidad, lo que facilita el trabajo de los desarrolladores al no perder tiempo en aprender a usar un controlador de versiones diferente.

Gestor de base de datos

Tecnología	Descripción	Ventajas	Desventajas
MySQL	Es uno de los gestores de bases de datos open-source más ampliamente usados, conocido por su confiabilidad, simplicidad y velocidad. Provee soporte robusto para muchos frameworks web.	- Bueno para aplicaciones web que requieren simplicidad y velocidad. - Tablas ilimitadas. - Bueno para aplicaciones con muchas lecturas. - Facilidad de uso con muchos frameworks web. - Seguridad robusta.	- 2^64 filas por tabla. - 64 índices por tabla. - 1 GB por query. - Más lento para aplicaciones con muchas escrituras. - Requiere soluciones third-party para escalabilidad de escritura.
MariaDB	Gestor de bases de datos robusto y open-source. Empezó como un fork de MySQL, pero ha evolucionado con funcionalidades adicionales. Es conocido por su compatibilidad con MySQL, pero con mejoras de performance y seguridad.	- Compatibilidad con MySQL con mejor escalabilidad. - Tablas ilimitadas. - Ejecución paralela de queries. - Excepcional para aplicaciones con muchas escrituras. - Escalable verticalmente. - Facilidad de uso con muchos frameworks web. - Funciones de seguridad un poco mejoradas que MySQL.	- 2^64 filas por tabla. - 64 índices por tabla. - 1 GB por query.
PostgreSQL	Muchas veces considerado como el gestor open-source más avanzado. Es excelente manejando queries complejos, asegurando la integridad de los datos y soportando funcionalidades extendibles.	- Bueno para proyectos grandes, data-intensive con queries complejos. - Filas ilimitadas por tabla. - Tablas ilimitadas. - Índices ilimitados por tabla. - Tamaño de query ilimitado. - Mejor para datasets grandes.	- Más lento en aplicaciones con muchas lecturas. - Más complejo que MariaDB y MySQL.

		- Muy escalable tanto vertical como horizontalmente. - Documentación extensa y comunidad grande. - Funciones de seguridad más avanzadas de entre las 3 opciones.	

(Abedinpour, 2024)

Selección final:

En base a la información encontrada, se seleccionó PostgreSQL como el gestor de bases de datos para el sistema MediNet. Esto se justifica por su mayor capacidad para manejar aplicaciones complejas, así como por sus avanzadas funcionalidades de integridad, escalabilidad y seguridad. A diferencia de MySQL y MariaDB, PostgreSQL ofrece soporte superior para consultas complejas, índices ilimitados por tabla y tamaño de consulta sin limitaciones prácticas, lo que lo convierte en una opción más adecuada para sistemas que requieren gestionar grandes volúmenes de información y operaciones simultáneas, como la coordinación de citas médicas entre miles de usuarios y cientos de clínicas. Además, PostgreSQL destaca por su fuerte enfoque en la integridad de los datos y por contar con funciones de seguridad más avanzadas, lo cual es especialmente relevante en un sistema que manejará información sensible de pacientes y agendas médicas. Por ejemplo, la Row Level Security que provee PostgreSQL permite que, en una misma tabla de citas, cada clínica solo vea sus datos basándose en el ID de usuario, garantizando privacidad y aislamiento de datos de forma nativa en el motor, lo cual es vital en el campo de salud. Aunque MySQL y MariaDB pueden resultar más simples o rápidas en ciertos escenarios con muchas lecturas o escrituras, PostgreSQL ofrece una mayor robustez, escalabilidad y flexibilidad a largo plazo, características que lo posicionan como la opción más adecuada para el crecimiento y la complejidad potencial del sistema MediNet.

Planificación e informe de gestión

Informe de gestión

Tarea	Quien la realizará	Fecha y hora de inicio	Fecha y hora de final
Informe de gestión	Todos	06/03/2026 10:00	06/03/2023 10:30
Hoja de presentación, Resumen e Introducción	Diego Gudiel	07/03/2026 10:00	07/03/2026 10:15
Prototipo 1, Presentación a usuarios y Lista de cambios	Luis Pedro Figueroa	07/03/2026 10:00	07/023/2026 16:00
Prototipo 2, Presentación a usuarios y Lista de cambios	Luis Pedro Hernández	07/03/2026 10:00	07/03/2026 16:00
Lista de requisitos funcionales	Santiago Cordero y Adair Velásquez	08/03/2026 10:00	08/03/2026 10:30
Diagrama y Descripción de clases	Luis Pedro Hernández y Luis Pedro Figueroa	08/03/2026 11:00	08/03/2026 13:00
Diagrama y descripción de paquetes	Adair Velásquez, Santiago Cordero y Diego Gudiel	09/03/2026 10:00	09/03/2026 12:00
Diagrama de clases persistentes	Santiago Cordero	10/03/2026 10:00	10/03/2026 11:00
Diagrama entidad-relación	Luis Pedro Hernández	10/03/2026 12:00	10/03/2026 13:00
Estimaciones de las dimensiones	Luis Pedro Figueroa y Adair Velásquez	11/03/2026 10:00	11/03/2026 12:30
Seleccionar tecnología	Luis Pedro Hernández y Santiago Cordero	12/03/2026 10:00	24/02/2026 13:00
Seleccionar gestor de base de datos	Diego Gudiel	12/03/2026 10:00	12/03/2026 11:00
Formularios LOGT	Todos	14/03/2026 12:00	14/03/2026 20:00

Conclusión de desempeño

Durante el periodo analizado, el equipo de desarrollo demostró un desempeño sólido y organizado. La distribución de tareas fue equitativa y cada miembro cumplió con sus responsabilidades dentro de los tiempos establecidos. Las interrupciones registradas fueron mínimas y en su mayoría justificadas, lo que refleja un buen nivel de enfoque y compromiso por parte de todos los integrantes. La colaboración entre miembros en tareas compartidas, como los diagramas y la selección de tecnología, evidencia una buena comunicación interna y capacidad de trabajo en equipo.

Sin embargo, se identificaron algunas áreas de mejora. En ciertas sesiones de trabajo prolongadas, los tiempos efectivos podrían optimizarse reduciendo las pausas no planificadas y estableciendo bloques de trabajo más estructurados. Asimismo, se recomienda que los comentarios registrados en las bitácoras sean más detallados, ya que esto facilitaría la trazabilidad del progreso y la identificación temprana de obstáculos en futuros ciclos de desarrollo.

Formularios LOGT

Diego Gudiel

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Tarea	Comentarios
06/03/2026	10:00	10:30	0	30min	Informe de gestión	Sin inconvenientes
07/03/2026	10:00	10:15	0	15min	Hoja de presentación, Resumen e Introducción	Completado a tiempo
09/03/2026	10:00	12:00	20 min	1h 40min	Diagrama y descripción de paquetes	Breve pausa para comer
12/03/2026	10:00	11:00	5 min	55min	Seleccionar gestor de base de datos	Revisión rápida de opciones
14/03/2026	12:00	12:15	0	15min	Formularios LOGT	Sin inconvenientes

Luis Pedro Figueroa

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Tarea	Comentarios

06/03/2026	10:00	10:30	0	30min	Informe de gestión	Sin inconvenientes
07/03/2026	10:00	16:00	60 min	5h 15min	Prototipo 1, Presentación a usuarios y Lista de cambios	Pausa para almuerzo y descanso
11/03/2026	10:00	12:30	10 min	2h 20min	Estimaciones de las dimensiones	Pequeña interrupción externa
14/03/2026	19:45	20:00	0	15min	Formularios LOGT	Sin inconvenientes

Luis Pedro Hernández

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Tarea	Comentarios
06/03/2026	10:00	10:30	0	30min	Informe de gestión	Sin inconvenientes
07/03/2026	10:00	16:00	45 min	5h 15min	Prototipo 2, Presentación a usuarios y Lista de cambios	Pausa para almuerzo
08/03/2026	11:00	13:00	0	2h 00min	Diagrama y Descripción de clases	Trabajo continuo
10/03/2026	12:00	13:00	10 min	50min	Diagrama entidad-relación	Consulta breve al equipo
12/03/2026	10:00	13:00	15 min	2h 45min	Seleccionar tecnología	Pausa para descansar
14/03/2026	14:00	14:15	0	15min	Formularios LOGT	Sin inconvenientes

Santiago Cordero

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Tarea	Comentarios
06/03/2026	10:00	10:30	0	30min	Informe de gestión	Sin inconvenientes
08/03/2026	10:00	10:30	0	30min	Lista de requisitos funcionales	Completado sin pausas
09/03/2026	10:00	12:00	0	2h 00min	Diagrama y descripción de paquetes	Trabajo fluido
10/03/2026	10:00	11:00	5 min	55min	Diagrama de clases persistentes	Revisión con compañero
12/03/2026	10:00	13:00	15 min	2h 45min	Seleccionar tecnología	Pausa por llamada
14/03/2026	14:00	14:15	0	15min	Formularios LOGT	Sin inconvenientes

Adair Velásquez

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Tarea	Comentarios
06/03/2026	10:00	10:30	0	30min	Informe de gestión	Sin inconvenientes
08/03/2026	10:00	10:30	0	30min	Lista de requisitos funcionales	Completado a tiempo
09/03/2026	10:00	12:00	15 min	1h 45min	Diagrama y descripción de paquetes	Consulta con compañero
11/03/2026	10:00	12:30	10 min	2h 20min	Estimaciones de las dimensiones	Pequeña distracción
14/03/2026	15:00	15:20	0	20min	Formularios LOGT	Trabajo rápido

Bibliografía

Abedinpour, P. (2024). MariaDB vs MySQL vs PostgreSQL vs SQLite: A Comprehensive Comparison for Web Applications.

https://medium.com/@peymaan.abedinpour/mariadb-vs-mysql-vs-postgresql-vs-sqlite-a-comprehensive-comparison-for-web-applications-0523cc3bc9d8

Becerril-Montekio, V & López-Dávila, L. (2011). Sistema de salud de Guatemala.

https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S0036-36342011000800015

Bubobot. (2025). Understanding the Ideal Uptime Rate for Websites.

https://dev.to/tomcao2012/understanding-the-ideal-uptime-rate-for-websites-i97

Cardinali, A. (2023). Demystifying the Math in Erlang-C Traffic Model for Non-Mathematicians.

https://www.soon.works/blog/erlang-c-traffic-model-for-non-mathematicians

DAE. (2022). Boletín estadístico afiliación 2022.

https://www.igssgt.org/wp-content/uploads/2023/11/Boletin-EstadIstico-de-Afiliacion-2022-IGSS.pdf

Essential Designs Team. (2025). Real-Time Data Sync: Best Practices for Cloud Apps.

https://www.essentialdesigns.net/news/real-time-data-sync-best-practices-for-cloud-apps

Geeksforgeeks. (2025). Git Vs. Other Version Control Systems.

https://www.geeksforgeeks.org/git/git-vs-other-version-control-systems-why-git-stands-out/

Index. (2026). Go vs Node.js vs FastAPI.

https://www.index.dev/skill-vs-skill/backend-go-vs-nodejs-vs-python-fastapi

INE. (2018). Características generales de la población.

https://censo2018.ine.gob.gt/censo2018/poblacion.php

INE. (2023). Compendio estadístico con enfoque de juventud.

https://www.ine.gob.gt/sistema/uploads/2023/11/06/20231106222842pWf6BcBWj8taVS3Q3mRKxgDsvwPejgH8.pdf

ReactiveProgramming. (2021). Arquitectura Cliente-Servidor.

https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/cliente-servidor

Simplilearn. (2025). Flutter vs Angular vs React: The Ultimate Showdown.

https://www.simplilearn.com/flutter-vs-angular-vs-react-article

Valle, C. (2020). Verdadero: en Guatemala hay 1.2 médicos por cada 1,000 habitantes.

https://lalinterna.agenciaocote.com/2023/08/09/verdadero-en-guatemala-hay-1-2-medicos-por-cada-1000-habitantes/

Worldometer. (2026). Población de Guatemala.

https://www.worldometers.info/es/poblacion-mundial/poblacion-guatemala/